MODULE IV(CONT.): FILE MANAGEMENT

INTRODUCTION

- A computer file is used to store data in digital format like plain text, image data or any other content.
- Computer files can be considered as the digital counterpart of paper documents, which traditionally are kept in office.
- While doing programming, you keep your source code in text files with different extensions.
- For example, C programming files have .c extension, Java programming files have .java extension.

File Operations

1) Creating a new file
2) Opening an existing file
3) Reading from and writing information to a file
4) Closing a file

DEFINING, OPENING AND CLOSING OF FILES

Defining a File

- While working with file, you need to declare a pointer of type 'FILE'. This declaration is needed for communication between file and program.

```
FILE *ptr;
```

Opening a File

- fopen( ) function can be used
  to create a new file or
  to open an existing file
- This function will initialize an object of the type FILE, which contains all the information necessary to control the stream.
- The syntax is shown below:

```
FILE *fopen( const char *filename, const char *access_mode );
```

where filename is string literal, which you will use to name your file.
access_mode can have one of the following values:

Mode	Description
r	Opens an existing text file for reading purpose
w	Opens a text file for writing, if it does not exist then a new file is created
a	Opens a text file for writing in appending mode
r+	Opens a text file for reading and writing both

Closing a File

- The file should be closed after reading/writing of a file.
- fclose( ) function can be used to close a file.
- The syntax is shown below:

```
int fclose(FILE *fp);
```

- The fclose( ) function returns zero on success or
  returns EOF(special character) if there is an error

For Solved Question Papers of UGC-NET/GATE/SET/PGCET in Computer Science, visit http://victory4sure.weebly.com/

INPUT AND OUTPUT OPERATIONS

Writing a File: fputc() fputs() & fprintf()

- You can use the following function to write individual characters to a stream:
  int fputc(int c, FILE *fp);
- The function fputc() writes the character value of argument c to the output stream referenced by fp.
- This function returns the written character on success; otherwise returns EOF if there is an error.
- You can use the following function to write a null-terminated string to a stream:
  int fputs(const char *s, FILE *fp);
- The function fputs() writes the string s into the file referenced by fp.
- This function returns a non-negative value on success; otherwise returns EOF if there is an error.
- You can use following function to write a string into a file:
  int fprintf(FILE *fp, const char *format, ...)

```
#include <stdio.h>
void main()
{
   FILE *fp;
   fp = fopen("/tc/bin/test.txt", "w+");
   fprintf(fp, "This is testing for fprintf...\n");
   fputs("This is testing for fputs...\n", fp);
   fclose(fp);
}
```

- When the above code is compiled and executed, it creates a new file test.txt in /tc/bin directory and writes two lines using two different functions.

Reading a File: fgetc() fgets() & fscanf()

- You can use the following function to read a text file character by character:
  int fgetc(FILE * fp);
- The fgetc() function reads a character from the input file referenced by fp.
- This function returns the character being read on success; otherwise returns EOF if there is an error.
- You can use the following function to read a string from a stream:
  char *fgets(char *buf, int n, FILE *fp);
- The functions fgets() reads up to n-1 characters from the input stream referenced by fp.
- It copies the read string into the buffer buf, appending a null character to terminate the string.
- If this function encounters a newline character '\n', then it returns only the characters read up to that point including new line character.
- You can also use int fscanf(FILE *fp, const char *format, ...) function to read strings from a file but it stops reading after the first space character encounters.

```
#include <stdio.h>
void main()
{
   FILE *fp;
   char buff[255];
   fp = fopen("/tc/bin/test.txt", "r");
   fscanf(fp, "%s", buff);
   printf("1 : %s \n", buff );
   fgets(buff, 255, (FILE*)fp);
   printf("2: %s \n", buff );
   fgets(buff, 255, (FILE*)fp);
   printf("3: %s ", buff );
   fclose(fp);
}
```

Output:

```
1 : This
2: is testing for fprintf...
3: This is testing for fputs...
```

- When the code is compiled and executed, it reads the file 'test.txt' in /tc/bin/ directory and produces the above result.

MODULE V: POINTERS & PREPROCESSORS

INTRODUCTION

• As you know, every variable is a memory-location and every memory-location has its address defined which can be accessed using ampersand(&) operator, which denotes an address in memory.

• Consider the following example, which will print the address of the variables defined:

```
#include<stdio.h>
void  main()
{
        int var1;
        char var2;
        printf("Address of var1 variable: %d \n", &var1 );
        printf("Address of var2 variable: %d", &var2 );
        return 0;
}
```

Output:

```
Address of var1 variable: 1266
Address of var2 variable: 1268
```

POINTER

• A pointer is a variable which holds address of another variable or a memory-location.

• For ex:

```
c=300;
pc=&c;
```

Here pc is a pointer; it can hold the address of variable c

& is called reference operator

DECLARATION OF POINTER VARIABLE

- Dereference operator(*) are used for defining pointer-variable.
- The syntax is shown below:

```
data_type *ptr_var_name;
```

- For ex:

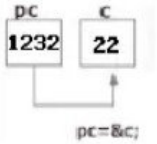
```
int *a;        // a as pointer variable of type int
float *c;      // c as pointer variable of type float
```

- Steps to access data through pointers:
  1) Declare a data-variable — ex: int c;
  2) Declare a pointer-variable — ex: int *pc;
  3) Initialize a pointer-variable — ex: pc=&c;
  4) Access data using pointer-variable — ex: printf("%d",*pc);
- Example: Program to illustrate working of pointers.

```
#include<stdio.h>
void  main()
{
  int *pc;
  int c;
  c=22;
  printf("Address of c: %d \n", &c);
  printf("Value of c: %d  \n", c);
  pc=&c;
  printf("Address of pointer pc: %d \n", pc);
  printf("Content of pointer pc: %d",*pc);
}
```

Output:

```
Address of c: 1232
Value of c: 22
Address of pointer pc: 1232
Content of pointer pc: 22
```



Explanation of Program and Figure

- Code int* pc; creates a pointer pc and
  code int c; creates normal variable c
- Code c=22; makes the value of c equal to 22, i.e., 22 is stored in the memory-location of variable c.
- Code pc=&c; makes pointer, point to address of c. Note that, &c is the address of variable c (because c is normal variable) and pc is the address of pc (because pc is the pointer-variable).
- Since the address of pc and address of c is same, *pc will be equal to the value of c.

NULL POINTER

- A NULL pointer is defined as a special pointer value that points to '\0'(nowhere) in the memory. In other words, NULL pointer does not point to any part of the memory.
- For ex:

```
int *p=NULL;
```


For Solved Question Papers of UGC-NET/GATE/SET/PGCET in Computer Science, visit http://victory4sure.weebly.com/

POINTERS AND ARRAYS

• Consider an array:

int arr[4];

• The above code can be pictorially represented as shown below:

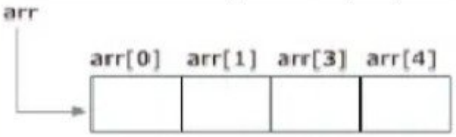


Figure: Array as Pointer

• The name of the array always points to the first element of an array.

• Here, address of first element of an array is &arr[0].

• Also, arr represents the address of the pointer where it is pointing. Hence, &arr[0] is equivalent to arr.

• Also, value inside the address &arr[0] and address arr are equal. Value in address &arr[0] is arr[0] and value in address arr is *arr. Hence, arr[0] is equivalent to *arr.

• Similarly,

&a[1] is equivalent to (a+1) AND, a[1] is equivalent to *(a+1).
&a[2] is equivalent to (a+2) AND, a[2] is equivalent to *(a+2).
&a[3] is equivalent to (a+1) AND, a[3] is equivalent to *(a+3).
.
.
&a[i] is equivalent to (a+i) AND, a[i] is equivalent to *(a+i).

• You can declare an array and can use pointer to alter the data of an array.

• Example: Program to access elements of an array using pointer.

```
#include<stdio.h>
void main()
{
  int data[5], i;
  printf("Enter elements: ");
  for(i=0;i<5;++i)
        scanf("%d", data+i);
  printf("You entered: ");
  for(i=0;i<5;++i)
        printf("%d   ",*(data+i) );
}
```

Output:

```
Enter elements:  1 2 3 5 4
You entered:  1 2 3 5 4
```

• Example: Program to find sum of all the elements of an array using pointers.

```
#include<stdio.h>
void main()
{
        int i, a[10], n, sum=0;
        printf("Enter the size of the array:");
        scanf("%d", &n);
        printf("Enter the elements into the array: ");
        for(i=0;i<n;i++)
             scanf("%d ",&a[i]);
        for(i=0;i<n;i++)
          sum+=*(a+i);
        printf("Sum --> %d ",sum);
}
```

Output:

```
Enter the size of the array:5
Enter the elements into the array: 2 4 6 10 15
Sum --> 37
```

POINTERS AND FUNCTIONS

- When, argument is passed using pointer, address of the memory-location is passed instead of value.
- Example: Program to swap 2 number using call by reference.

```
#include<stdio.h>

void swap(int *a,int *b)
{   // pointer a and b points to address of num1 and num2 respectively
    int temp;
    temp=*a;
    *a=*b;
    *b=temp;
}

void  main()
{
  int num1=5,num2=10;
  swap(&num1, &num2);     //address of num1 & num2 is passed to swap function
  printf("Number1 = %d \n",num1);
  printf("Number2 = %d",num2);
}
```

Output:

```
Number1 = 10
Number2 = 5
```

Explanation

- The address of memory-location num1 and num2 are passed to function and the pointers *a and *b accept those values.
- So, the pointer a and b points to address of num1 and num2 respectively.
- When, the value of pointer is changed, the value in memory-location also changed correspondingly.
- Hence, change made to *a and *b was reflected in num1 and num2 in main function.
- This technique is known as call by reference.

For Solved Question Papers of UGC-NET/GATE/SET/PGCET in Computer Science, visit http://victory4sure.weebly.com/

POINTER ARITHMETIC

- As you know, pointer is an address, which is a numeric value.
- Therefore, you can perform arithmetic operations on a pointer just as you can a numeric value.
- There are 4 arithmetic operators that can be used on pointers: ++, --, +, and –
- To understand pointer arithmetic, let us consider that ptr is an integer pointer which points to the address 1000.
- Assuming 16-bit integers, let us perform the following arithmetic operation on the pointer:

  ptr++
- Now, after the above operation, the ptr will point to the location 1002 because each time ptr is incremented, it will point to the next integer location which is 2 bytes next to the current location.

Incrementing a Pointer

- We prefer using a pointer in our program instead of an array because the variable pointer can be incremented, unlike the array name which cannot be incremented because it is a constant pointer.
- Example: Program to increment the variable pointer to access each succeeding element of the array.

```
#include<stdio.h>
void main()
{
        int var[] = {10, 100, 200};
        int i, *ptr;
        ptr = var;
        for ( i = 0; i < 3; i++)
        {
                printf("Address of var[%d] = %x \n", i, ptr );
                printf("Value of var[%d] = %d \n", i, *ptr );
                ptr++;              //move to the next location
        }
}
```

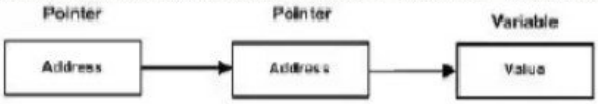
Output:

```
Address of var[0] = 1130
Value of var[0] = 10
Address of var[1] = 1132
Value of var[1] = 100
Address of var[2] = 1134
Value of var[2] = 200
```


For Solved Question Papers of UGC-NET/GATE/SET/PGCET in Computer Science, visit http://victory4sure.weebly.com/

POINTERS TO POINTERS

• A variable which contains address of a pointer-variable is called pointer to a pointer.



• A variable that is a pointer to a pointer must be declared as such. This is done by placing an additional asterisk in front of its name.

• For example, following is the declaration to declare a pointer to a pointer of type int:

```
int **var;
```

• When a target value is indirectly pointed to by a pointer to a pointer, accessing that value requires that the asterisk operator be applied twice, as is shown below in the example:

```
#include<stdio.h>
void main()
{
        int var;
        int *ptr;
        int **pptr;
        var = 3000;         // take the address of var
        ptr = &var;         // take the address of ptr using address of operator &
        pptr = &ptr;        // take the value using pptr
        printf("Value of var = %d \n", var );
        printf("Value available at *ptr = %d \n", *ptr );
        printf("Value available at **pptr = %d ", **pptr);
        return 0;
}
```

Output:

```
Value of var = 3000
Value available at *ptr = 3000
Value available at **pptr = 3000
```


For Solved Question Papers of UGC-NET/GATE/SET/PGCET in Computer Science, visit http://victory4sure.weebly.com/

PREPROCESSOR

• A preprocessor is not part of the compiler, but is a separate step in the compilation process.
• In simplistic terms, a preprocessor is just a text substitution tool and they instruct compiler to do required pre-processing before actual compilation.
• All preprocessor commands begin with a pound symbol(#).
• List of pre-processor directives:

#include
This is used insert a particular header from another file.
#define, #undef
These are used to define and un-define conditional compilation symbols.
#if, #elif, #else, #endif
These are used to conditionally skip sections of source code.
#error
This is used to issue errors for the current program.

Use of #include

• Let us consider very common preprocessing directive as below:
#include <stdio.h>
• Here, "stdio.h" is a header file and the preprocessor replace the above line with the contents of header file.
• Example: Program to illustrate use of #include

```
#include <stdio.h>
int main()
{
   printf("VTUNOTESBYSRI");
   return 0;
}
```

Output:
VTUNOTESBYSRI

Use of #define

• Analyze following examples to understand this directive.
#define PI 3.1415
• The string 3.1415 is replaced in every occurrence of symbolic constant PI,
• Example: Program to find area of a circle using #define.

```
#include<stdio.h>
#define PI 3.1415
int main()
{
    int radius;
    float area;
    printf("Enter the radius: ");
    scanf("%d", &radius);
    area=PI*radius*radius;
    printf("Area=%.2f",area);
    return 0;
}
```

Output:
Enter the radius: 3
Area=28.27

For Solved Question Papers of UGC-NET/GATE/SET/PGCET in Computer Science, visit http://victory4sure.weebly.com/

Use of #if, #elif, #else and #endif

• The preprocessor directives #if, #elif, #else and #endif allows to conditionally compile a block of code based on predefined symbols.

• Example: Program to illustrate this concept.

```
#include<stdio.h>
#define MAX  100
void main()
{
        #if(MAX)
                printf("MAX is defined");
        #else
                printf ("MAX is not defined");
        #endif
}
```

Output:

```
MAX is defined
```

Use of #error

• The #error directives allow instructing the compiler to generate an error.

• For example, we wish to issue a warning when a symbol is defined in the current project.

• Example: Program to illustrate use of #error.

```
#include<stdio.h>
#define MAX  100
void main()
{
        #if(MAX)
        #error: MAX is defined by me!!!
        . .
        . .
        . .
        #endif
}
```

Output:

```
#error: MAX is defined by me!!!
```

Use of #undef

• Consider an example shown below:

```
#undef FILE_SIZE
#define FILE_SIZE 42
```

• This tells the preprocessor to undefine existing FILE_SIZE and define it as 42.

For Solved Question Papers of UGC-NET/GATE/SET/PGCET in Computer Science, visit http://victory4sure.weebly.com/

MEMORY ALLOCATION FUNCTIONS

• There are 2 types:

1) Static Memory Allocation:

• If memory-space to be allocated for various variables is decided during compilation-time itself, then the memory-space cannot be expanded to accommodate more data or cannot be reduced to accommodate less data.

• In this technique, once the size of the memory-space to be allocated is fixed, it cannot be altered during execution-time. This is called static memory allocation.

• For ex,

```
int a[5];
```

2) Dynamic Memory Allocation

• Dynamic memory allocation is the process of allocating memory-space during execution-time i.e. run time.

• If there is an unpredictable storage requirement, then the dynamic allocation technique is used.

• This allocation technique uses predefined functions to allocate and release memory for data during execution-time.

• There are 4 library functions for dynamic memory allocation:

1) malloc()
2) calloc()
3) free()
4) realloc()

• These library functions are defined under "stdlib.h"

malloc()

- The name malloc stands for "memory allocation".
- This function is used to allocate the requirement memory-space during execution-time.
- The syntax is shown below:

```
data_type *p;
p=(data_type*)malloc(size);
        here p is pointer variable
            data_type can be int, float or char
            size is number of bytes to be allocated
```

- If memory is successfully allocated, then address of the first byte of allocated space is returned.
  If memory allocation fails, then NULL is returned.
- For ex:

```
ptr=(int*)malloc(100*sizeof(int));
```

- The above statement will allocate 200 bytes assuming sizeof(int)=2 bytes.
- Example: Program to find sum of n elements entered by user. Allocate memory dynamically using malloc() function.

```
#include <stdio.h>
#include <stdlib.h>
void main()
{
   int n, i, *ptr, sum=0;
   printf("Enter number of elements: ");
   scanf("%d",&n);
   ptr=(int*)malloc(n*sizeof(int));     //memory allocated using malloc
   printf("Enter elements of array: ");
   for(i=0;i<n;++i)
   {
      scanf("%d  ",ptr+i);
      sum+=*(ptr+i);
   }
   printf("Sum=%d",sum);
   free(ptr);
}
```

Output:

```
Enter number of elements: 3
Enter elements of array: 2  5  1
Sum= 8
```


For Solved Question Papers of UGC-NET/GATE/SET/PGCET in Computer Science, visit http://victory4sure.weebly.com/

calloc()

• The name calloc stands for "contiguous allocation".
• This function is used to allocate the required memory-size during execution-time and at the same time, automatically initialize memory with 0's.
• The syntax is shown below:

```
data_type *p;
p=(data_type*)calloc(n,size);
```

• If memory is successfully allocated, then address of the first byte of allocated space is returned.
If memory allocation fails, then NULL is returned.
• The allocated memory is initialized automatically to 0's.
• For ex:

```
ptr=(int*)calloc(25,sizeof(int));
```

• The above statement allocates contiguous space in memory for an array of 25 elements each of size of int, i.e., 2 bytes.
• Example: Program to find sum of n elements entered by user. Allocate memory dynamically using calloc() function.

```
#include <stdio.h>
#include <stdlib.h>
void main()
{
   int n,i,*ptr,sum=0;
   printf("Enter number of elements: ");
   scanf("%d",&n);
   ptr=(int*)calloc(n,sizeof(int));
   printf("Enter elements of array: ");
   for(i=0;i<n;++i)
   {
      scanf("%d  ",ptr+i);
      sum+=*(ptr+i);
   }
   printf("Sum=%d",sum);
   free(ptr);
}
```

Output:

```
Enter number of elements: 3
Enter elements of array: 2  5  1
Sum= 8
```

free()

• Dynamically allocated memory with either calloc() or malloc() does not get return on its own. The programmer must use free() explicitly to release space.
• The syntax is shown below:

```
free(ptr);
```

• This statement causes the space in memory pointed by ptr to be deallocated.

For Solved Question Papers of UGC-NET/GATE/SET/PGCET in Computer Science, visit http://victory4sure.weebly.com/

realloc()

• If the previously allocated memory is insufficient or more than sufficient. Then, you can change memory-size previously allocated using realloc().

• The syntax is shown below:

ptr=(data_type*)realloc(ptr,newsize);

• Example: Program to illustrate working of realloc().

```
#include <stdio.h>
#include <stdlib.h>
void main()
{
   int *ptr, i, n1, n2;
   printf("Enter size of array: ");
   scanf("%d",&n1);
   ptr=(int*)malloc(n1*sizeof(int));
   printf("Address of previously allocated memory: ");
   for(i=0;i<n1;i++)
       printf("%u \n", ptr+i);
   printf("\n Enter new size of array: ");
   scanf("%d",&n2);
   ptr=realloc(ptr,n2);
   printf("Address of newly allocated memory: ");
   for(i=0;i<n2;i++)
       printf("%u \n", ptr+i);
}
```

Output:

```
Enter size of array: 3
Address of previously allocated memory:
1022
1024
1026
Enter new size of array: 5
Address of newly allocated memory:
1022
1024
1026
1028
1030
```


For Solved Question Papers of UGC-NET/GATE/SET/PGCET in Computer Science, visit http://victory4sure.weebly.com/